

现代通用操作系统的动态链接、内存管理与页面缓存重定向机制深度研究报告

现代操作系统架构演进与底层抽象的趋同性

现代通用操作系统 (Modern General-Purpose Operating Systems)，以Linux、Windows和macOS为绝对主导，在历经数十年的架构演进后，展现出了高度的底层设计趋同性。尽管这三大操作系统在应用程序编程接口 (API)、可执行文件格式以及内核层数据结构的具体实现上存在显著差异，但它们都受制于相同的冯·诺依曼计算机体系结构以及现代微处理器的物理限制。因此，为了实现高效的资源利用、严格的进程隔离以及极致的输入输出 (I/O) 性能，它们不可避免地采用了在逻辑抽象上极度相似的核心机制。

本研究报告将系统性地深入剖析三大操作系统中的三个核心底层机制，并以此直接回应系统架构设计与底层安全领域的关键疑问。首先，本报告将探讨动态链接与加载机制，验证Linux中以共享对象和动态链接器为代表的机制是否在Windows和macOS中具有同构的实现。其次，本报告将详细论述虚拟地址与物理地址的映射体系，分析现代操作系统如何利用硬件内存管理单元实现地址空间的虚拟化与隔离。最后，本报告将深入研究页面缓存 (Page Cache) 机制的核心原理，并结合最新的学术研究与顶级安全会议论文，深度剖析如何通过修改底层页面缓存或内存映射结构，实现在文件访问时将其无痕重定向至其他物理页面的高级内核态攻击与控制技术。

现代操作系统的动态链接与加载机制的跨平台演进

在早期的计算机软件工程中，静态链接 (Static Linking) 是构建可执行程序的唯一方式。在静态链接模型下，链接器在编译阶段将所有依赖的目标文件和库文件中的指令与数据，完整地复制并合并到一个庞大且独立的可执行文件中¹。这种范式虽然赋予了程序极强的环境独立性并极大缩短了启动时间，但其劣势同样致命：如果多个正在运行的进程同时依赖同一个基础库（例如C标准库中的格式化输出函数），该库的机器码将在物理内存中被复制多份，造成极其严重的内存资源和磁盘空间浪费²。为了解决这一工程瓶颈，现代操作系统全面引入并高度依赖动态链接 (Dynamic Linking) 机制。

动态链接的核心哲学是推迟符号解析 (Symbol Resolution) 与地址重定位 (Relocation) 的时间窗口，将其从静态的编译时推迟到程序的加载时 (Load Time) 乃至运行时 (Run Time)¹。尽管Linux、Windows和macOS在这一机制的具体实现细节上各有千秋，但它们均提供了一个专门的系统级组件——动态链接器或加载器——来完成这一复杂的时空转换任务。

Linux体系：ELF格式、PLT/GOT机制与延迟绑定

在Linux生态系统中，动态链接库通常被称为共享对象 (Shared Objects)，其文件扩展名为常见的特定后缀，采用可执行与可链接格式 (Executable and Linkable Format, ELF)¹。当Linux内核启动一个依赖共享对象的动态链接程序时，操作系统的加载过程并不会在内核态直接结束，而是将控制权临时移交给驻留在用户态的动态链接器¹。这个动态链接器负责在虚拟地址空间中映射程序所需的各个共享对象，并修正程序中对外部函数和全局变量的引用⁴。

Linux动态链接体系中一个极具代表性且在性能优化上至关重要的设计是延迟绑定(Lazy Binding)机制。在实际的大型软件运行场景中,一个程序可能会链接数十个乃至上百个共享对象,包含成千上万个外部符号。如果在程序启动时由动态链接器一次性解析所有符号的真实物理内存映射地址,将产生不可接受的启动延迟⁵。为了实现延迟绑定,ELF格式在数据段和代码段中分别引入了全局偏移表(Global Offset Table, GOT)和过程链接表(Procedure Linkage Table, PLT)⁵。

当程序首次调用一个外部共享库中的函数时,指令流并不会直接跳转到目标函数的真实地址,而是首先跳转到该函数在代码段中对应的PLT表项⁵。此时,由于函数尚未被解析,PLT表项中的指令会从GOT表中读取一个默认地址,该地址实际上指向了PLT表内部的下一条指令,进而触发对动态链接器内部符号查找函数的调用⁵。动态链接器随即在已加载的共享对象模块树中执行字符串匹配与哈希查找,一旦定位到目标函数的真实内存基址,便将该绝对地址回填至GOT表中对应的条目中⁵。当程序在后续执行中再次调用该函数时,指令流经由PLT表将直接从GOT表中读取到已修正的真实内存地址,从而实现以极低开销完成跨模块调用⁵。此外,为了确保共享库可以在虚拟内存的任意基址处被加载而无需修改其只读代码段, Linux编译器普遍采用特定选项生成位置无关代码(Position-Independent Code, PIC),利用相对于程序计数器的偏移量进行寻址⁷。

Windows体系:PE格式、输入地址表与预先绑定策略

在微软的Windows操作系统中,动态链接机制基于可移植可执行(Portable Executable, PE)文件格式,其核心产物被称为动态链接库(Dynamic-Link Library, DLL)²。虽然在逻辑目标上与Linux高度一致,但Windows的NT加载器(NT Loader)和链接器在设计哲学上采用了更为静态的前期绑定策略,且对符号的导出与导入管理有着截然不同的结构规范。

在Windows环境下编译可执行文件时,如果需要链接DLL中的函数,微软的工具链通常依赖一个导入库(Import Library)作为编译期的“胶水”⁹。这个导入库并不包含实际的执行代码,而是包含了目标DLL的导出表(Export Table)的元数据映射关系⁹。在PE文件内部,操作系统维护着一个被称为输入地址表(Import Address Table, IAT)的关键数据结构。与Linux中高度依赖运行时动态解析的PLT/GOT延迟绑定机制不同,当Windows的PE加载器将程序及其依赖的DLL映射到虚拟内存时,加载器会主动遍历PE文件中的导入目录,利用底层的等价机制依次定位每个DLL模块,并一次性或在模块初始化阶段将所有被调用的外部函数的真实内存地址填入IAT中³。

这种预先填充IAT的机制带来了两个显著的特征:其一,由于DLL支持通过函数名称或独有的序号(O ordinal)导出符号,按序号导出曾被广泛用于隐藏未公开的系统内部API⁹;其二,由于IAT的内存结构在程序启动后便已确定且处于用户态地址空间,它成为了Windows环境下恶意软件与高级安全研究人员进行系统调用拦截最经典的攻击平面¹⁰。通过在内存中篡改目标进程的IAT表项(即IAT Hooking技术),攻击者可以轻易将合法程序的API调用重定向至恶意构造的负载模块中,这在早期的第二代Rootkit开发中极为盛行¹⁰。

macOS体系:Mach-O格式、dyld与命名空间隔离

苹果生态系统中的macOS及其衍生操作系统采用了Mach-O(Mach Object)文件格式,并配备了专有的动态链接编辑器(dyld)²。与Linux所采用的System V ABI标准中基于平坦命名空间(Flat Namespace)的设计不同, macOS在动态链接符号解析上引入了极为严格的双层命名空间(Two-Level Namespace)机制¹³。

在传统的平坦命名空间中，所有的全局符号都被置于同一个虚拟的解析池中。如果一个进程加载的多个动态库中存在同名的函数，动态链接器通常会按照库加载的顺序决定使用哪一个，这极易引发符号冲突和不可预期的劫持行为¹³。而macOS的双层命名空间则要求在编译时，调用方不仅记录所需的符号名称，还必须将该符号与其来源的特定动态库(Dylib)名称强行绑定¹⁴。在运行时，dyld仅会在指定的动态库中检索该符号，从根本上消除了大型软件工程中的符号名称空间污染问题¹⁴。尽管如此，为了向后兼容传统的UNIX软件，macOS依然允许开发者通过特定的链接器标志退化至平坦命名空间模式¹⁵。

此外，为了应对现代图形化操作系统中海量基础库加载对启动速度的拖累，macOS创新性地实现并广泛应用了共享缓存(dyld shared cache)技术¹⁷。操作系统在更新或初始化时，会将数百个系统核心动态库(如libSystem和各类UI框架)预先链接、重定位并合并为一个巨大的二进制缓存文件¹⁷。在进程启动时，该共享缓存被统一映射到所有用户态进程的受保护地址空间中，极大降低了逐个加载和解析库文件的内存抖动与时间开销¹⁷。然而，高级用户与安全研究人员仍可通过配置特定环境变量，强制加载自定义动态库以实现合法的函数拦截与执行流重定向¹⁶。

动态链接机制特性	Linux操作系统	Windows操作系统	macOS操作系统
可执行文件标准格式	ELF (Executable and Linkable Format)	PE (Portable Executable)	Mach-O
动态库文件扩展名	.so (Shared Object)	.dll (Dynamic-Link Library)	.dylib (Dynamic Library)
系统核心动态加载器	ld-linux.so / ld.so	NT Loader / ntdll.dll	dyld
核心符号解析与地址重定位结构	过程链接表(PLT) 与 全局偏移表(GOT)	输入地址表(IAT) 与 导出表(EAT)	动态库加载命令(LC_LOAD_DYLIB)与共享缓存
符号命名空间设计哲学	默认采用平坦命名空间 (Flat Namespace)	基于DLL边界边界的导出隔离与运行时解析	默认采用双层命名空间 (Two-Level Namespace)

运行时动态库预加载与拦截机制	环境变量机制	注册表项与底层IAT内存篡改 (IAT Hooking)	环境变量机制与内核底层的拦截重定位
----------------	--------	------------------------------	-------------------

综上所述，尽管三大操作系统在二进制结构、链接器组件和符号解析策略上各自演化，但“动态加载机制是否为现代通用操作系统共有”的答案是确凿无疑的。它们均建立在推迟内存地址绑定的理论之上，旨在最大化物理内存利用率并实现系统核心库的模块化更新。

虚拟内存与物理内存映射机制及地址空间隔离体系

在现代操作系统的设计范式中，进程的执行空间不能直接暴露于底层的物理硬件上。查询中提及“Linux中有虚拟地址、物理地址，其他操作系统是不是也有”，这一推论精确地切中了现代计算体系架构的核心命题。如果没有虚拟内存 (Virtual Memory) 机制，所有的程序都将直接操作物理 RAM 的绝对地址，这不仅会导致多任务环境下的内存覆写灾难，更使得操作系统无法执行复杂的内存管理策略，如按需分页 (Demand Paging) 和内存交换 (Swapping) ²⁰。

硬件支持与内存管理单元 (MMU) 的协作

虚拟地址 (亦称为逻辑地址) 是中央处理器 (CPU) 在执行应用程序指令时所生成和引用的地址空间，它是完全被抽象和虚构的 ²⁰。而物理地址则代表了主板上随机存取存储器 (RAM) 芯片中实际存储数据的硅晶片单元物理位置 ²⁰。在用户态运行的程序永远无法直接看到或访问其自身数据的物理地址 ²⁰。

为了在这两个世界之间建立桥梁，现代 CPU 架构 (无论是 x86_64、ARM64 还是 RISC-V) 都集成了一个至关重要的硬件协处理器——内存管理单元 (Memory Management Unit, MMU) ²¹。当 CPU 发出读写某个虚拟地址的指令时，MMU 会以极高的频率拦截这一请求，并查阅操作系统在物理内存中精心维护的映射字典——即页表 (Page Tables) ²⁴。

由于现代 64 位系统的地址空间极其广阔，单层页表将占用无法估量的物理内存。因此，操作系统普遍采用多级树状页表结构来进行地址转换 ²⁵。然而，执行一次完整的内存读取可能需要遍历 4 到 5 级页表，产生多次额外的内存访问 (Page Walk)，这会导致严重的性能衰退 ²⁵。为缓解这一冯·诺依曼瓶颈，处理器内部集成了转换后备缓冲器 (Translation Lookaside Buffer, TLB)，专门用于高速缓存近期已完成转换的虚拟页到物理页帧的映射关系 ²⁶。硬件的 MMU/TLB 机制与操作系统的页表管理代码构成了现代计算机内存隔离与保护的绝对基石。

跨平台的虚拟地址空间管理与隔离策略

在 Linux 操作系统中，64 位架构下的每个进程都产生一种错觉，认为自己独占了极度庞大且连续的虚拟地址空间 ²²。Linux 内核通过进程控制块中的特定数据结构，利用多级基数树严格维护着从全局页目录 (PGD) 到具体页表项 (PTE) 的层级关系 ²⁵。当应用程序通过系统调用申请内存时，Linux 并不会立即分配昂贵的物理内存，而仅仅是在内核空间中分配一个虚拟内存区域 (VMA) ²⁹。直到应用程序首次尝试读写这块虚拟内存时，MMU 才会因找不到映射而触发一次缺页异常 (Page Fault)。此时，Linux 的缺页异常处理程序接管执行，从操作系统的空闲页框链表中分配一个真实

的物理页，更新对应的页表项，随后恢复用户进程的执行³⁰。当系统物理内存面临枯竭时，内核后台线程则会评估页面的使用频率，将不活跃的匿名页面交换(Swap)至磁盘上的交换分区，从而腾出物理空间³²。

Windows操作系统在内存虚拟化上采用了与Linux在逻辑上高度同构的设计架构。在传统的32位Windows系统中，每个进程默认被分配2GB的用户态私有虚拟地址空间以及2GB的系统内核共享空间；而在64位系统架构下，这一空间上限被扩展到了惊人的128TB²⁴。Windows的内存管理器(Memory Manager)负责监控和分配虚拟页(通常同样为4KB大小)，并管理各个进程的工作集(Working Set)³⁴。当物理内存产生压力时，Windows会执行工作集修剪机制，将长期未被访问的物理页转移至备用列表(Standby List)中³⁵。如果内存资源进一步紧缺，这些页面会被彻底写入磁盘上的页面文件(Pagefile.sys)，实现物理内存的超额分配与流转²⁴。

苹果的macOS其内核XNU则采用了融合式设计。在虚拟内存层面，它深刻继承了Mach微内核的VM子系统理念³⁷。Mach VM摒弃了传统的线性页表思维，转而采用高度抽象的虚拟内存对象(vm_object)来表示每一块连续的内存区域³⁸。每一个分配给进程的虚拟内存对象最终都会被映射到底层的通用页面列表(Universal Page List, UPL)上³⁹。UPL充当了Mach微内核与底层硬件MMU页表架构之间的翻译官，负责调度物理页帧的调入与调出操作³⁹。

内存管理系统特征	Linux内存子系统	Windows内存管理器	macOS (Mach VM)
底层硬件映射依赖	内存管理单元 (MMU) 与 TLB	内存管理单元 (MMU) 与 TLB	内存管理单元 (MMU) 与 TLB
虚拟内存分配粒度	虚拟内存区域 (VMA)	虚拟地址分配单元 (Virtual Alloc)	虚拟内存对象 (vm_object)
物理页管理核心结构	物理页框描述符 (struct page)	物理帧数据库 (PFN Database)	通用页面列表 (UPL)
内存交换与页面淘汰机制	页面置换守护进程与交换分区	备用列表修剪与页面文件	虚拟内存分页器与交换文件
缺页异常处理策略	延迟分配与按需调页	按需调页与工作集动	Mach缺页异常消息传

	(Demand Paging)	态调整	递机制
--	-----------------	-----	-----

无论是Linux的VMA架构、Windows的内存管理器还是macOS的Mach VM，其核心目标都完全一致：通过在虚拟地址和物理地址之间引入一层坚不可摧的间接映射抽象，现代通用操作系统才得以实现进程沙箱化、防止越权数据访问、消解物理内存碎片，并为后续更高级的文件与内存融合机制（即页面缓存）奠定基础。

页面缓存 (Page Cache) 机制的底层架构与跨平台实现

在确立了虚拟内存与物理内存之间存在映射关系后，我们进入操作系统的又一核心设计——页面缓存 (Page Cache)。正如用户在查询中深刻指出的，Linux中的页面缓存负责维护文件在存储设备上的位置与系统物理页之间的映射关系。现代计算机体系中，无论是机械硬盘的寻道延迟还是固态硬盘的物理擦写开销，其速度都比主存 (RAM) 慢若干个数量级⁴¹。如果每次文件读取都必须发起底层的直接存储设备 I/O，整个系统的计算效能将发生灾难性的阻塞。

因此，所有现代通用操作系统的内核都将未被应用程序堆栈显式占用的空闲物理内存接管过来，用作透明的软件级缓存区⁴¹。这就使得存储在磁盘上的文件数据被预先读取或保留在内存中。只要有足够的空闲内存，操作系统就尽可能长时间地将文件数据留在内存里，这就是页面缓存机制的由来⁴¹。

Linux的页面缓存：从Radix Tree到XArray的数据结构进化

在Linux内核中，页面缓存最初的功能是为了加速虚拟文件系统 (VFS) 中系统调用对文件数据的访问效率⁴³。为了高效率地管理这种缓存，Linux构建了一个名为地址空间 (address_space) 的结构体，用于在特定文件 (由inode表示) 与被缓存在内存中的物理页框之间建立严密的跟踪网络⁴²。

对于页面缓存而言，查找操作的频率极高。当进程试图读取文件的某个偏移量时，内核必须以微秒级的速度确认该偏移量对应的数据是否已经存在于某个物理页中⁴²。在过去的十几年中，Linux内核采用了一种高度压缩的字典树结构——基数树 (Radix Tree) 来维持这一映射关系⁴²。基数树允许内核利用文件内的逻辑偏移量作为键值，快速遍历树节点，进而获取关联的物理页指针。

然而，随着服务器硬件向超多核架构发展，基数树在并发访问控制上的锁争用问题日益凸显。因此，在内核开发者的主导下，Linux底层架构经历了一次重大的重构，基数树被彻底替换为一种更为先进的数据结构——XArray⁴⁶。XArray在逻辑表现上类似于一个动态调整大小的巨型指针数组，但其内部深度集成了读取-复制-更新 (RCU) 无锁同步机制⁴⁸。如今，当Linux用户态程序执行内存映射文件时，VFS层将直接无锁地查询XArray⁴⁶；如果目标页存在于缓存中，则命中缓存并直接通过修改页表将该物理页映射到进程的虚拟空间；若未命中，内核会分配新的物理页框，发起底层块设备 I/O 读取数据填充页面，最后将该新页面的指针原子性地插入XArray中⁴⁶。

Windows的缓存管理器：VACB、共享缓存映射与原型PTE

尽管Linux的实现广为人知，但Windows操作系统同样拥有一套逻辑上高度对等且设计极为精巧

的模块, 被称为缓存管理器(Cache Manager)⁵⁰。Windows的缓存管理器并不孤立运作, 而是与Windows内存管理器以及文件系统驱动程序形成了一个紧密咬合的“三驾马车”架构, 共同实现虚拟块缓存机制⁵¹。

在Windows中, 当一个文件被首次开启且允许被缓存时, 内核会为该文件流分配并初始化一个核心数据结构: 共享缓存映射(SharedCacheMap)⁵²。这个结构在功能定位上与Linux的address_space如出一辙, 是整个文件数据在内存中存在的枢纽⁵²。SharedCacheMap结构内部维护着一个极为关键的数组——视图访问控制块(View Access Control Block, VACB)数组³⁴。为了规避缓存碎片化并提高管理效率, Windows不会按单页进行缓存映射, 而是将大文件按256KB的固定粒度划分为多个逻辑槽位(Slots/Views)³⁴。当发生缓存未命中时, 缓存管理器会查找或分配一个空闲的VACB, 指导内存管理器将256KB大小的文件数据视图载入到系统内核的工作集中³⁴。

为了确保文件数据在整个操作系统层面(即使是被多个不相关进程同时访问)物理上只存在唯一的一份拷贝, Windows内存管理器利用段对象(Section Object)机制维持单实例性(Single Instancing)⁵²。段对象内部通过一种名为原型页表项(Prototype PTE, PPTE)的数据结构来实现物理页帧的间接映射³⁴。PPTE驻留在系统级非分页池中, 当不同进程映射同一个文件时, 它们各自进程页表中的虚拟PTE并不会直接指向物理内存, 而是作为指针链指向系统的PPTE, 最终由PPTE指向存放文件数据的单一物理缓存页³⁴。

macOS的统一缓冲区缓存(UBC): vnode与VM对象的深度融合

在操作系统发展的早期, BSD系统的文件系统块缓存与虚拟内存管理的页缓存是相互割裂的, 导致同一文件的数据可能在物理内存中保留两份不同的副本。为了彻底解决这一冗余与同步冲突, 苹果公司在macOS(及前身Darwin)中全面采用了统一缓冲区缓存(Unified Buffer Cache, UBC)架构³⁸。

在macOS的XNU内核中, 代表文件系统抽象节点的核心结构是vnode³⁸。当一个文件被打开或访问时, 内核会确保该文件的vnode关联一个ubc_info结构体³⁸。ubc_info结构充当了极其关键的桥梁, 它包含了一个指向Mach虚拟内存子系统中对应的内存对象(vm_object)的指针³⁸。

通过UBC机制的绑定, 所有针对文件内容的I/O读写调用以及直接的内存映射调用, 最终都会被路由到同一个虚拟内存对象上。底层Mach VM的分页器(Pager)在调度内存时, 使用通用页面列表(Universal Page List, UPL)机制来确定哪些物理页框正在缓存该文件的哪一部分数据³⁹。这确保了无论应用层使用何种接口, macOS底层始终维持着文件系统与物理内存地址之间绝对唯一的映射路由³⁹。

页面缓存篡改与文件访问重定向的攻击平面与技术实现

基于上述对三大操作系统内存抽象架构的论述, 我们终于可以正面回应查询中最深刻的问题: “通过修改页面缓存, 从而让访问该文件时被重定向到其他物理页, 这种机制在其他OS中是否也存在或能达到类似效果?”

从操作系统底层安全架构的角度来看, 答案是肯定的, 且这一机制正是现代高级持续性威胁(APT)、高级Rootkit程序以及内核层漏洞利用技术所倚仗的核心攻击平面。由于现代通用操作系统在

文件访问链条上必然经过“文件描述对象 -> 树状/数组索引结构 -> 物理页或页表映射”这一层级抽象，只要攻击者获取了操作系统的最高特权级(Ring 0)，并掌握了直接内核对象操控(Direct Kernel Object Manipulation, DKOM)的技术，就能在不修改磁盘上实际文件内容的前提下，在内存中动态劫持并篡改这一映射关系，从而实现访问重定向和彻底的内存隐形(Memory Cloaking)¹⁰。

Linux环境:XArray覆盖与Dirty Pagetable页表劫持技术

在Linux内核中，针对页面缓存实施重定向的理论依据已经被学术界和工业界多次证实。根据相关学术论文研究，利用页面缓存纯粹是操作系统共享的软件状态这一特征，未授权进程可以通过侧信道时序测量，探测并推断目标物理页在缓存中的状态，进而构建极高精度的时序攻击(测时精度在纳秒级徘徊)⁵⁹。

若要在Linux内核中将特定文件的访问重定向至恶意的物理页框，安全研究人员通常依赖于两种核心的映射覆写技术：

第一种是直接篡改页面缓存的底层数据结构。如前所述，Linux的address_space利用XArray来维护映射⁴²。内核级Rootkit可以通过DKOM技术定位到特定监控文件(如系统审计日志或核心二进制文件)对应的inode，进而解析并遍历其关联的XArray节点。攻击者只需将XArray中特定的物理页指针(struct page *)替换为指向攻击者预先控制且注入了恶意负载的匿名物理页。此后，任何合法的安全审计软件尝试读取该文件时，VFS层将基于被篡改的XArray信息，直接将攻击者的恶意物理页数据返回给安全软件，实现无懈可击的文件欺骗⁴²。

第二种是利用最新漏洞利用工程中频繁出现的Dirty Pagetable技术框架(例如在CVE-2024-1086中展示的跨内核版本提权技术)⁶²。这种技术不直接修改文件缓存结构，而是利用内核中的Use-After-Free或堆溢出漏洞，制造物理页的内存重叠，进而使得攻击者能够直接控制进程的页表结构(Page Directory/PTE)²⁸。通过主动改写PTE项中的物理页框号(PFN)并伪造适当的权限位，攻击者可以在无需向磁盘写入任何恶意文件(即Fileless执行)的条件下，使正常的系统调用或程序指令在进行地址转换时，被强制且隐蔽地重定向到攻击者控制的内存区域²⁸。

Windows环境:原型PTE操纵、FILE_OBJECT劫持与TLB去同步攻击

在Windows系统生态中，由于缓存管理器、对象管理器和内存管理器的强耦合设计，针对文件访问的重定向劫持呈现出更为复杂的攻击路径。各类高级Rootkit(如DirtyMoe)及APT攻击均大量采用此类DKOM技巧在内核深处实现隐身与权限维持⁶³。

首先是在内存映射层面的原型PTE篡改。鉴于Windows系统利用段对象(Section Object)中的原型PTE(PPTE)来实现所有进程对特定文件的缓存共享³⁴，恶意内核驱动可以通过定位系统内特定系统保护文件的PPTE结构，将其映射的物理地址覆写。这种劫持的影响力是全局性的，它将导致整个操作系统内所有试图加载或读取该文件的进程，被系统内存管理器统一重定向至黑客指定的物理内存页，从而实现全局DLL劫持或执行流替换³⁴。

其次是在对象管理器层面的句柄与FILE_OBJECT劫持。当Windows应用程序使用特定的API打开文件时，系统中的安全参考监视器(Security Reference Monitor)通常只在文件打开(如ZwCreateFile阶段)时进行访问控制检查⁶⁶。在获得句柄后，底层的读取和写入操作不再经过严苛的安全校验⁶⁶。恶意代码可以定位到Windows内核中的对象句柄表，直接篡改句柄所引用的

FILE_OBJECT数据结构中的关键指针字段(如FsContent等)⁶⁶。通过这种底层内存对象劫持,即使用户态程序请求读取正常的受保护文件,底层的文件系统解析逻辑也会被欺骗,导致最终的访问被重定向至磁盘上或缓存中一个完全不同的秘密文件⁵⁸。

更为极致的是由著名安全研究员提出的Shadow Walker(影子漫步者)第三代Rootkit所展示的内存层面的重定向技术¹⁰。由于现代处理器的内存管理单元将用于执行指令的TLB(iTLB)和用于读取数据的TLB(dTLB)在硬件层面上分离开来,Shadow Walker巧妙地拦截并控制了Windows操作系统的缺页异常处理流程²⁶。当系统的反病毒扫描软件尝试“读取”被感染的内核内存时,Rootkit通过篡改页表让MMU返回伪造的、充满合法零数据的物理页;而当CPU实际“执行”该地址的指令时,异常处理程序又将其重定向回包含真实恶意代码的物理页²⁶。这种通过操纵硬件TLB导致操作系统内部页表视图分裂的技术,提供了在现代OS中进行底层物理页面重定向的最经典范例⁶⁷。

macOS环境: vnode直接操控与动态重定向机制

在苹果操作系统的封闭生态中,基于XNU内核的底层结构依然未能免疫文件映射关系被篡改和重定向的风险。这种劫持可以根据权限边界分为用户态的环境注入与内核态的对象深层篡改两种层次。

在用户态环境下,由于macOS的双层命名空间和专有的动态加载器dyld机制,攻击者可以通过配置特定环境变量(如DYLD_INSERT_LIBRARIES)或使用工具链特性,强制将自定义的恶意的动态库注入到正在运行或即将启动的目标进程中¹⁶。利用macOS独有的Interposing(拦截/重定位)特性,恶意动态库可以在内存中轻易覆盖并重定向标准系统文件操作函数(例如open、read乃至底层硬件交互的ioctl系统调用)的原始解析逻辑⁷⁰。这使得文件读取操作可以在用户态被透明地代理至其他文件或虚构的内存缓冲区域,而主程序对此毫无察觉⁷⁰。

当攻击场景延伸至操作系统的内核态(即绕过系统完整性保护SIP,加载恶意内核扩展KEXT时),文件访问重定向则变成了彻头彻尾的内核对象操纵对抗³⁷。在此层面上,macOS内核表示文件和目录实体的核心数据结构为vnode。恶意内核驱动可以通过提取系统中的内核任务端口(例如利用特定的特权调用如task_for_pid(0)),获得遍历并操控系统内部所有进程与文件对象结构的超级权限⁷²。一旦定位到目标审计日志文件或敏感系统二进制对应的vnode实例,Rootkit便可通过覆盖与该vnode相关联的操作向量表或回调函数(例如关键的路径解析函数VNOP_LOOKUP)⁵⁶。由于vnode不可撤销地绑定了代表缓存状态的ubc_info结构,这也就意味着攻击者阻断或替换了文件描述符到达其真实的底层缓存物理页的路径,将系统的I/O和映射流向强制重定向至虚假的内存结构中⁴⁰。

恶意修改缓存与映射 重定向技术概览	Linux内核对抗技术	Windows内核对抗技术	macOS内核对抗技术
缓存与结构数据替换	XArray页指针覆写定位至恶意物理页	篡改段对象与原型页表项 (PPTE)	篡改vnode结构与VNOP_LOOKUP重定向

底层页表对象劫持机制	内存重叠与脏页表 (Dirty Pagetable) 修改PFN	修改进程工作集PTE与物理内存覆盖	修改内核任务数据结构与UPL列表干扰
对象级权限逃逸与代理	修改inode及虚拟文件系统对象描述符	直接内核对象操作篡改FILE_OBJECT句柄	内核扩展获取task_for_pid(0)实施DKOM
高级缓存隔离与欺骗攻击	页面缓存侧信道测量攻击 (Page Cache Attacks)	TLB去同步化 (TLB Desynchronization / Splitting)	用户态dyld共享缓存注入与跨库拦截调用

学术前沿与经典参考文献深度剖析

为了提供充分的计算机科学理论支撑并深刻验证上述针对页面缓存和物理地址重定向的探讨，以下详细剖析数篇在系统架构演进和顶级网络安全会议中发表的关键学术论文及研究成果。这些文献构筑了现代操作系统底层缓存管理机制与内核态重定向攻防理论的坚实学术框架。

1. 《**Page Cache Attacks**》(发表于2019年系统安全顶级会议 **ACM CCS**) 该学术论文由安全学者Daniel Gruss、Ari Trachtenberg等人联合发表，创造性地将操作系统的纯软件缓存(即页面缓存)作为高精度侧信道攻击的暴露目标⁵⁹。这项研究通过跨平台对比证明，由于Linux(底层利用树状缓存结构)与Windows(利用缓存管理器机制)在系统全局范围内透明且积极地将文件映射并保留在物理页中，攻击者能够通过精密的时序探测手段测量内存映射文件的读取延迟⁶¹。实验数据表明，在Linux系统上此探测的时间分辨率高达2微秒，而在Windows 10系统上更低至466纳秒的极限水平⁶¹。这篇文献在学术上无可辩驳地证实了：尽管操作系统存在异构性，但维护文件与物理页缓存映射机制在现代通用操作系统中是完全等价且普遍存在的底层功能，这也为针对缓存实施侧信道监控和重定向篡改奠定了理论基石⁵⁹。
2. 《**Shadow Walker: Raising The Bar For Rootkit Detection**》(首次发布于**Black Hat**安全会议) 这篇极具开创性的安全研究报告由系统安全专家Sherri Sparks与Jamie Butler联合撰写，其详细定义了高级内存隐蔽和物理页面重定向的第三代Rootkit范式¹⁰。由于当时操作系统的内存管理器普遍盲目信任硬件在进行地址转换时返回的物理页表数据，该Rootkit架构创新性地实现了名为“TLB去同步化 (TLB Desynchronization / Splitting)”的底层技术²⁶。通过在内核层级接管并挂钩缺页异常处理器 (Page Fault Handler)，Shadow Walker在底层强行让数据读取请求和代码执行请求解析到截然不同的物理内存页²⁶。这种技术使得操作系统的文件缓存连同安全扫描器所查看到的系统视图彻底失效，从而向业界提供了关于“拦截系统地址转换以将访问重定向至任意物理页”的最极端的概念证明²⁶。

3. 《Windows Kernel Hijacking Is Not An Option: MemoryRanger Comes to the Rescue Again》(发表于 JDFSL 2021 期刊) 该学术论文由系统安全学者Igor Korkin教授撰写, 系统性地揭示了Windows操作系统内核在保护动态分配数据结构方面的固有脆弱性⁶⁶。研究明确指出, 对象管理器所维护的关键结构(如系统级的Handle Table和极其核心的FILE_OBJECT结构体)非常容易遭受底层劫持⁶⁶。一旦未经授权的恶意内核驱动程序能够在内存中定位并修补诸如FsContent等文件对象内部字段指针, 它便能够在完全绕过Windows操作系统内置安全监视器(Security Reference Monitor)的前提下, 实现文件内容读取逻辑的无痕重定向⁶⁶。论文同时提出并验证了利用虚拟化技术构建安全飞地(Enclaves)的理论防线, 通过将被怀疑的内核驱动程序隔离以切断其直接操纵页缓存和内核对象的路径, 从防御角度再次印证了这种底层重定向攻击的存在与破坏力⁶⁶。
4. 《UBC: An Efficient Unified I/O and Memory Caching Subsystem for NetBSD》(发表于2000年 USENIX Annual Technical Conference) 由核心开发者Chuck Silvers撰写的这篇经典系统架构论文, 详尽、透彻地阐释了统一缓冲区缓存(Unified Buffer Cache, UBC)的设计思想及其工程实现⁴⁰。该架构不仅是NetBSD内核改造的基石, 其核心设计理念至今仍深植于苹果macOS系统内核之中。论文详细分析了早期UNIX操作系统中离散处理块设备缓存和虚拟内存缓存所带来的双重冗余与数据不一致性困境⁴⁰。通过UBC架构的理论确立, 系统级开发者达成了广泛共识: 现代操作系统的文件数据在内存中必须以与虚拟内存系统管理匿名页框完全相同的方式(即基于页面的寻址模式), 并通过维护同一个底层的中心化映射表(如macOS继承的通用页面列表机制UPL)进行全局一致的管理调度³⁹。

结论与体系架构层面的终极思考

通过对现代三大通用主流操作系统——Linux、Windows与macOS在底层架构和内存处理抽象模型的全面对比与深度剖析, 我们可以提炼出针对计算机系统体系结构的系列决定性结论:

第一, 现代操作系统的演进轨迹高度受制于硬件架构的物理极限与计算效能法则。这促使不同的系统内核在解决内存消耗、启动延迟以及存储设备读写瓶颈时, 衍生出了在逻辑抽象上高度同构的解决方案。动态链接机制中的地址解析推迟思想, 虚拟内存中的隔离与按需分配策略, 无一不彰显着操作系统对硬件机制(如MMU和TLB)的深度榨取。

第二, 针对查询中提出的“维护文件对应物理页映射关系并进行修改重定向”的深度疑问, 本报告的分析明确证实: 所有现代通用操作系统不仅都不可或缺地包含用于维持此类关系的统一页面缓存(Page Cache)机制, 而且这种映射网络不论是依赖Linux的XArray树形结构、Windows中极其严密的VACB插槽与PTE原型页表项组合, 还是macOS中高度耦合的UBC与UPL链路, 其在本质上均构成了一个决定数据生死流向的“内核级中心化路由表”。

第三, 正由于这套页面缓存与映射解析网络承载着操作系统最核心的地址转换和性能加速重任, 它不可逆转地成为了恶意软件开发者、高级持续性威胁组织以及系统级漏洞研究人员所争夺的最终攻防咽喉要道。从Linux环境中直接进行指针覆写与“脏页表”(Dirty Pagetable)漏洞提权操作, 到Windows系统中通过内核级对象操纵实现原型PTE篡改与文件句柄劫持, 再到通过硬件TLB特性进行缓存分割导致执行流重定向的极限技术, 这些在学术研究和工业应用中被反复验证的技术实践深刻昭示: 通过获取特权手段干预和篡改操作系统底层的“文件-内存”映射关系网络, 进而重定向物理页帧以欺骗系统调用, 在理论依据上是确凿无疑的, 在跨平台的操作实践中更已成为内核安全对抗的常态范式。所有现代操作系统的高效性均深深根植于这种底层的地址抽象转换架构

之中, 因此, 若安全防御体系无法确保底层对象指针链与页表结构的绝对完整性, 试图在应用层拦截这种基于深层缓存重定向的高级伪装攻击, 无疑是徒劳且无效的。

引用的著作

1. How the Dynamic Linker Works (and How Binaries from Other Languages Link Together) | by Matiltheory | Medium, 访问时间为 四月 15, 2026, <https://medium.com/@matiltheory/how-the-dynamic-linker-works-and-how-binaries-from-other-languages-link-together-f5dd90d2d571>
2. Dynamic linker - Wikipedia, 访问时间为 四月 15, 2026, https://en.wikipedia.org/wiki/Dynamic_linker
3. Differences between dynamically linked libraries (.dll) and shared objects (.so), 访问时间为 四月 15, 2026, <https://www.jwhitham.org/2017/10/dll.html>
4. Dynamic Linking? How does that work? : r/C_Programming - Reddit, 访问时间为 四月 15, 2026, https://www.reddit.com/r/C_Programming/comments/1lmehvu/dynamic_linking_how_does_that_work/
5. PLT and GOT Workflow in Dynamic Linking - openEuler, 访问时间为 四月 15, 2026, <https://www.openeuler.org/en/blog/lijiajie128/2020-11-10-%E5%8A%A8%E6%80%81%E9%93%BE%E6%8E%A5%E4%B8%AD%E7%9A%84PLT%E4%B8%8EGOT>
6. Linking Explained Static vs Dynamic vs LTO Guide in Modern C++, 访问时间为 四月 15, 2026, https://simplifycpp.org/books/minibooklet/mini_booklet_Linking_Explained_Static_vs_Dynamic_vs_LTO_Guide.pdf
7. Discussion on the Dynamic-link Library Mechanism under Linux - Alibaba Cloud Community, 访问时间为 四月 15, 2026, <https://www.alibabacloud.com/blog/602169>
8. `Static` vs `Dynamic` library in `Linux`, `PIC`, `PIE`, `plt`, `got`, `Virtual vs Physical Addresses`, 访问时间为 四月 15, 2026, <https://gist.github.com/reveng007/b9ef8c7c7ed7a46b10a325f4dee42ac4>
9. c++ - Dynamic linking - Linux Vs. Windows - Stack Overflow, 访问时间为 四月 15, 2026, <https://stackoverflow.com/questions/58603797/dynamic-linking-linux-vs-windows>
10. "SHADOW WALKER" Raising The Bar For Rootkit Detection - Black Hat, 访问时间为 四月 15, 2026, <https://blackhat.com/presentations/bh-jp-05/bh-jp-05-sparks-butler.pdf>
11. Hijack Execution Flow, Technique T1574 - Enterprise | MITRE ATT&CK®, 访问时间为 四月 15, 2026, <https://attack.mitre.org/techniques/T1574/>
12. Choosing the Right OS: Mac, Windows, or Linux — A Programmer's Guide! - Medium, 访问时间为 四月 15, 2026, <https://medium.com/@nomannayeem/choosing-the-right-os-mac-windows-or-linux-a-programmers-guide-2d128c40fc73>
13. I've been wondering about qualitative differences between Mach-O and ELF, after, 访问时间为 四月 15, 2026, <https://news.ycombinator.com/item?id=19931229>

14. Compiling Your Code in OS X - Apple Developer, 访问时间为 四月 15, 2026,
<https://developer.apple.com/library/archive/documentation/Porting/Conceptual/PortingUnix/compiling/compiling.html>
15. What does "expected in: flat namespace" mean? - Stack Overflow, 访问时间为 四月 15, 2026,
<https://stackoverflow.com/questions/32277351/what-does-expected-in-flat-namespace-mean>
16. Mac OS X Manual Page For dyld(1), 访问时间为 四月 15, 2026,
<https://leopard-adc.pepas.com/documentation/Darwin/Reference/ManPages/man1/dyld.1.html>
17. Reversing iOS System Libraries Using Radare2: A Deep Dive into Dyld Cache (Part 1), 访问时间为 四月 15, 2026,
<https://www.nowsecure.com/blog/2024/09/11/reversing-ios-system-libraries-using-radare2-a-deep-dive-into-dyld-cache-part-1/>
18. A Deep Dive into Penetration Testing of macOS Applications (Part 3) - CyberArk, 访问时间为 四月 15, 2026,
<https://www.cyberark.com/resources/threat-research-blog/a-deep-dive-into-penetration-testing-of-macos-applications-part-3>
19. Injecting functions into binaries with two-level namespaces · blueboxd chromium-legacy · Discussion #211 - GitHub, 访问时间为 四月 15, 2026,
<https://github.com/blueboxd/chromium-legacy/discussions/211>
20. Difference between physical/logical/virtual memory address - Stack Overflow, 访问时间为 四月 15, 2026,
<https://stackoverflow.com/questions/15851225/difference-between-physical-logical-virtual-memory-address>
21. Difference Between Virtual and Physical Addresses - Tutorialspoint, 访问时间为 四月 15, 2026,
<https://www.tutorialspoint.com/article/difference-between-virtual-and-physical-addresses>
22. Logical vs Physical Address in Memory Management : r/compsci - Reddit, 访问时间为 四月 15, 2026,
https://www.reddit.com/r/compsci/comments/35e0zz/logical_vs_physical_addresses_in_memory_management/
23. Memory paging - Wikipedia, 访问时间为 四月 15, 2026,
https://en.wikipedia.org/wiki/Memory_paging
24. Virtual Address Spaces - Windows drivers | Microsoft Learn, 访问时间为 四月 15, 2026,
<https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/virtual-address-spaces>
25. A Novel Technique for Performance-Aware Orchestration of Thread and Page Table Placement in NUMA Systems - arXiv, 访问时间为 四月 15, 2026,
<https://arxiv.org/html/2502.10923v2>
26. Memory Analysis Against Anti-Forensics Methods | by Alireza Taghikhani - Medium, 访问时间为 四月 15, 2026,
<https://medium.com/@alirezataghikhani1998/memory-analysis-against-anti-foren>

[sics-methods-22dd2c37f2a](#)

27. Understanding Page Tables, Swapping, and Memory Allocation | Linux Journal, 访问时间为 四月 15, 2026,
<https://www.linuxjournal.com/content/linux-memory-management-understanding-page-tables-swapping-and-memory-allocation>
28. Kernel Exploitation Techniques: Turning The (Page) Tables - sam4k, 访问时间为 四月 15, 2026, <https://sam4k.com/page-table-kernel-exploitation/>
29. Exporting Kernel Page Caching for Efficient User-Level I/O, 访问时间为 四月 15, 2026, <https://www.fsl.cs.stonybrook.edu/docs/msst10export/msst10export.pdf>
30. Understanding and troubleshooting page faults and memory swapping - Site24x7, 访问时间为 四月 15, 2026,
<https://www.site24x7.com/learn/linux/page-faults-memory-swapping.html>
31. Swapping in Operating Systems. How Operating Systems swap pages from... | by Humberto Filho | Medium, 访问时间为 四月 15, 2026,
https://medium.com/@humbertofilho_30158/swapping-in-operating-systems-7553faef944c
32. Chapter 11 Swap Management - The Linux Kernel Archives, 访问时间为 四月 15, 2026, <https://www.kernel.org/doc/gorman/html/understand/understand014.html>
33. How does the kernel decide between disk-cache vs swap? - Unix & Linux Stack Exchange, 访问时间为 四月 15, 2026,
<https://unix.stackexchange.com/questions/678806/how-does-the-kernel-decide-between-disk-cache-vs-swap>
34. Inside the Windows Cache Manager, 访问时间为 四月 15, 2026,
<https://mirror.gpmidi.net/vx/Papers/Windows/Internals%20and%20Analysis/2022-09-05%20-%20Inside%20the%20Windows%20Cache%20Manager.pdf>
35. Disable or flush page cache on Windows - Stack Overflow, 访问时间为 四月 15, 2026,
<https://stackoverflow.com/questions/22666042/disable-or-flush-page-cache-on-windows>
36. PSA: Your Ram is likely being claimed by "standby" in windows. This can cause micro and massive stuttering in games. - Reddit, 访问时间为 四月 15, 2026,
https://www.reddit.com/r/pcmasterrace/comments/r7c4fn/psa_your_ram_is_likely_being_claimed_by_standby/
37. macOS Rootkit Emulation - CyStack, 访问时间为 四月 15, 2026,
<https://cystack.net/research/kernel-rootkit-emulation>
38. Section 8.10. Unified Buffer Cache (UBC) | Mac OS X Internals - Flylib.com, 访问时间为 四月 15, 2026, <https://flylib.com/books/en/3.126.1.93/1/>
39. Managing Data - IOKit Fundamentals - Apple Developer, 访问时间为 四月 15, 2026,
<https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/IOKitFundamentals/DataMgmt/DataMgmt.html>
40. Chuck Silvers. UBC: An Efficient Unified I/O and Memory Caching Subsystem for NetBSD, 2000, 访问时间为 四月 15, 2026, <https://netbsd.stupin.su/en/ubc/>
41. Page cache - Wikipedia, 访问时间为 四月 15, 2026,
https://en.wikipedia.org/wiki/Page_cache

42. Life-long-Learner/linux-kernel-development/chapter-16.md at master - GitHub, 访问时间为 四月 15, 2026,
<https://github.com/firmianay/Life-long-Learner/blob/master/linux-kernel-development/chapter-16.md>
43. Difference between page cache and buffer cache - LinuxQuestions.org, 访问时间为 四月 15, 2026,
<https://www.linuxquestions.org/questions/linux-kernel-70/difference-between-page-cache-and-buffer-cache-636294/>
44. Why we use radix-tree(or xarray) for storing page caches? - Stack Overflow, 访问时间为 四月 15, 2026,
<https://stackoverflow.com/questions/62447084/why-we-use-radix-treeor-xarray-for-storing-page-caches>
45. Page Cache Management in Virtual Environments - Luddy Homepage Server, 访问时间为 四月 15, 2026,
<https://homes.luddy.indiana.edu/prateeks/papers/mtp.pdf>
46. Memory management Page cache Page frame reclaiming Swapping Swap cache, 访问时间为 四月 15, 2026,
https://students.mimuw.edu.pl/ZSO/Wyklady/10_memory3/PageCache-Reclaiming.pdf
47. Convert page cache to XArray - LWN.net, 访问时间为 四月 15, 2026,
<https://lwn.net/Articles/750540/>
48. XArray — The Linux Kernel documentation, 访问时间为 四月 15, 2026,
<https://www.kernel.org/doc/html/v5.8/core-api/xarray.html>
49. Chapter 10 Page Frame Reclamation - The Linux Kernel Archives, 访问时间为 四月 15, 2026,
<https://www.kernel.org/doc/gorman/html/understand/understand013.html>
50. Cache Manager, 访问时间为 四月 15, 2026,
https://ecs.syr.edu/faculty/fawcett/handouts/Seminar_BrownBag06/Chap11-Cache%20Manager.ppt
51. Inside the Cache Manager - ITPro Today, 访问时间为 四月 15, 2026,
<https://www.itprotoday.com/it-infrastructure/inside-the-cache-manager>
52. Windows Kernel Internals Cache Manager, 访问时间为 四月 15, 2026,
<https://www.i.u-tokyo.ac.jp/edu/training/ss/lecture/new-documents/Lectures/15-CacheManager/CacheManager.pdf>
53. Cache manager shared cache map - NTDEV - OSR Developer Community, 访问时间为 四月 15, 2026,
<https://community.osr.com/t/cache-manager-shared-cache-map/58334>
54. File Caching - Win32 apps - Microsoft Learn, 访问时间为 四月 15, 2026,
<https://learn.microsoft.com/en-us/windows/win32/fileio/file-caching>
55. UBC: AN EFFICIENT UNIFIED I/O AND MEMORY CACHING SUBSYSTEM FOR NET BSD - USENIX, 访问时间为 四月 15, 2026,
https://www.usenix.org/event/usenix2000/freenix/full_papers/silvers/silvers.pdf
56. vnode_isdir | Apple Developer Documentation, 访问时间为 四月 15, 2026,
https://developer.apple.com/documentation/kernel/1562276-vnode_isdir
57. Hooked on Linux: Rootkit Taxonomy, Hooking Techniques and Tradecraft - Elastic,

访问时间为 四月 15, 2026,

<https://www.elastic.co/security-labs/linux-rootkits-1-hooked-on-linux>

58. Demystifying Modern Windows Rootkits - YouTube, 访问时间为 四月 15, 2026,
<https://www.youtube.com/watch?v=ZASslpdumcY>
59. Page Cache Hacking | College of Engineering - Boston University, 访问时间为 四月 15, 2026, <https://www.bu.edu/eng/2019/01/30/page-cache-hacking/>
60. Accelerated Linux page cache attacks detailed | brief - SC Magazine, 访问时间为 四月 15, 2026,
<https://www.scworld.com/brief/accelerated-linux-page-cache-attacks-detailed>
61. Page Cache Attacks - Michael Schwarz, 访问时间为 四月 15, 2026,
<https://misc0110.net/files/pagecacheattacks.pdf>
62. Flipping Pages: An analysis of a new Linux vulnerability in nf_tables and hardened exploitation techniques - Pwning Tech, 访问时间为 四月 15, 2026,
<https://pwning.tech/nftables/>
63. What Is a Rootkit? - Palo Alto Networks, 访问时间为 四月 15, 2026,
<https://www.paloaltonetworks.com/cyberpedia/rootkit>
64. Rootkit, Technique T1014 - Enterprise | MITRE ATT&CK®, 访问时间为 四月 15, 2026,
<https://attack.mitre.org/techniques/T1014/>
65. DirtyMoe: Rootkit Driver - Gen Digital, 访问时间为 四月 15, 2026,
<https://www.gendigital.com/blog/insights/research/dirtymoe-rootkit-driver>
66. Windows Kernel Hijacking Is Not an Option: MemoryRanger Comes ..., 访问时间为 四月 15, 2026,
<https://commons.erau.edu/cgi/viewcontent.cgi?article=1726&context=jdfsl>
67. DEF CON 13 - Sherri Sparks & Jamie Butler, "Shadow Walker" - Raising The Bar For Rootkit Detection - YouTube, 访问时间为 四月 15, 2026,
<https://www.youtube.com/watch?v=Q89pjQ2kqow>
68. TLB;DR: Enhancing TLB-based Attacks with TLB Desynchronized Reverse Engineering - USENIX, 访问时间为 四月 15, 2026,
<https://www.usenix.org/system/files/sec22-tatar.pdf>
69. MoRE Shadow Walker: TLB-splitting on Modern x86 - Black Hat, 访问时间为 四月 15, 2026,
<https://blackhat.com/docs/us-14/materials/us-14-Torrey-MoRE-Shadow-Walker-The-Progression-Of-TLB-Splitting-On-x86-WP.pdf>
70. Practical macOS Security Researcher Notes and Guide (OSMR) - Zeyad Azima, 访问时间为 四月 15, 2026, <https://zeyadazima.com/notes/osmrnotes/>
71. Analyzing CVE-2024-44243, a macOS System Integrity Protection bypass through kernel extensions | Microsoft Security Blog, 访问时间为 四月 15, 2026,
<https://www.microsoft.com/en-us/security/blog/2025/01/13/analyzing-cve-2024-44243-a-macos-system-integrity-protection-bypass-through-kernel-extensions/>
72. Revisiting Mac OS X Kernel Rootkits - GitHub Gist, 访问时间为 四月 15, 2026,
<https://gist.github.com/zchee/5973634d4ebf1fef099ab6b434d41bfc>